EAST WEST UNIVERSITY

Department of Computer Science and Engineering

Project Report

on

Taka Gelo Koi — Where Did the Money Go?

Course Code	CSE 412
Course Title	Software Engineering
Section	02
Submitted To	Dr. Mohammad Mahdi Hassan

Submitted By: Team NewToDev (Team 17)

Student ID	Name	Role
2023-2-60-117	MD. Sifat Bin Islam	Team Lead
2023-2-60-176	Md. Arifur Rahman Shihab	Backend Developer
2023-2-60-294	Md. Sabbir Ahmed	Testing
2023-2-60-217	Adnan Anan Sikder	Frontend Developer

Submission Date: 19 April 2026

Table of Contents

Table of Contents 2
Chapter 1: Project Introduction 5
1.1 Introduction 5
1.2 Overview of the System 5
1.3 Scope and Objectives 6
1.4 Stakeholders 6
Chapter 2: Requirements 7
2.1 Requirements Collection Process 7
2.2 User Stories 7
2.3 Acceptance Criteria 8
US-01: Google Sign-In 8
US-02: Log a Transaction 8
US-03: Upload Receipt for OCR 8
US-04: Auto-Categorisation 8
US-07 and US-08: Monthly Budget and Alerts 8
US-09 and US-10: Income Tracking 9
US-13 to US-15: Savings and Goals 9
US-16 and US-17: Loan Management 9
2.4 Use Case Diagram 9
Chapter 3: Architecture and Detailed Design 10
3.1 System Architecture 10
3.1.1 Presentation Layer 10
3.1.2 Application Layer 10
3.1.3 Data Layer 10
3.2 Technology Stack 11
Chapter 4: Database Design 12
4.1 Overview 12
4.2 Entity Relationship Description 12
4.3 Table Definitions 12
4.4 Indexing Strategy 15
4.5 Entity Relationship Diagram 15
Chapter 5: Class Design 16
5.1 Overview 16
5.2 Backend Classes and Modules 16
5.2.1 Database Model Classes (models.py) 16
5.2.2 Pydantic Schema Classes (schemas.py) 16
5.2.3 OCR Service Module (ocr_service.py) 16
5.2.4 Parser Service Module (parser_service.py) 17

5.3 Frontend Components and Hooks 17
5.3.1 App.jsx 17
5.3.2 useApi.js (Custom Hook) 17
5.3.3 Page and Layout Components 18
5.4 Class Diagram 18
Chapter 6: Design Patterns 19
6.1 Overview 19
6.2 Factory Method Pattern (Creational) 19
Purpose 19
Application in Taka Gelo Koi 19
6.3 Adapter Pattern (Structural) 19
Purpose 19
Application in Taka Gelo Koi 20
Chapter 7: UI/GUI Design 22
7.1 Design Philosophy 22
7.2 Typography 22
7.3 Landing Page 22
7.4 Login Page 22
7.5 Dashboard 22
Sidebar 22
Overview Panel 22
Transactions Panel 23
Budgets Panel 23
Income Panel 23
Savings and Goals Panel 23
Analytics Panel 23
Report Panel 23
Loans Panel 23
7.6 Responsive Design 23
7.7 UI Screenshots 23
Chapter 8: Scheduling and Estimation 24
8.1 Project Timeline 24
8.2 KLOC Estimation 24
8.3 Function Point Estimation 25
8.4 Effort and Duration Estimation (COCOMO II) 25
8.5 Productivity Metrics 26
Chapter 9: Testing 27
9.1 Testing Strategy 27
9.2 Backend Test Coverage 27

Schema Validation Tests (test_schemas.py) 27
API Route Tests (test_api_routes.py) 27
9.3 Frontend Test Coverage 27
Component and Hook Tests (App.test.jsx) 27
9.4 Test Results Summary 28
9.5 Test Configuration 28
Appendix 29
A. Project Resources 29
B. Database Migration History 29
C. API Endpoint Reference 29
D. OCR Configuration 30

Chapter 1: Project Introduction

1.1 Introduction

Managing personal finances requires consistent effort, particularly in tracking day-to-day expenditures across multiple spending categories. For individuals in Bangladesh, this challenge is intensified by the widespread use of cash transactions alongside digital payment platforms such as bKash and Nagad, which often leave users without a consolidated view of their spending. Standard bank statements, where available, typically list transactions without contextual category information, making it difficult for users to understand their spending patterns in a meaningful way.

Taka Gelo Koi (loosely translated as "Where Did the Money Go?") is a web-based personal finance management application developed to address this problem. The system enables users to log, categorise, and analyse their daily expenditures through an interactive web interface. A key component of the system is its integration with the Google Cloud Vision API, which allows users to upload photographs of physical or handwritten receipts and have the relevant financial information — including the transaction amount, date, and merchant name — extracted automatically. This reduces the manual effort required for data entry and supports users who deal with handwritten or printed receipts in both Bengali and English.

Beyond expense tracking, Taka Gelo Koi provides a comprehensive financial management suite covering income recording, monthly budget management with threshold-based alerts, savings goal tracking with allocation from income or a pooled savings balance, and a loan management module for tracking borrowed money and repayments. A visual analytics dashboard presents spending trends through graphical summaries, giving users an immediate understanding of their financial behaviour.

The application is built using a React-based single-page application for the client side and a Python FastAPI backend for server-side logic. Authentication is handled through Google OAuth 2.0, which eliminates the need for a traditional username-and-password system. Data is persisted in a PostgreSQL relational database with nine tables covering all functional domains of the application.

1.2 Overview of the System

Taka Gelo Koi follows a three-tier web application architecture consisting of a presentation layer, an application layer, and a data layer. The presentation layer is a React-based single-page application that renders the landing page, login interface, and the main dashboard. The dashboard is organised into functional tabs covering transactions, budgets, income, savings and goals, analytics, monthly reports, and loan management.

The application layer is a FastAPI server that exposes a RESTful API. It handles Google OAuth 2.0 token verification, full CRUD operations for transactions, budgets, incomes, goals, goal allocations, savings transfers, loans, and loan payments. It also provides OCR processing via the Google Cloud Vision API, receipt text parsing, monthly summary computation, income summary, savings pool aggregation, and automated savings allocation. The data layer consists of a PostgreSQL database with nine primary tables.

The OCR pipeline operates as follows: a user uploads a photograph of a receipt through the dashboard. The backend saves the file and calls the Google Cloud Vision API using the DOCUMENT_TEXT_DETECTION feature, which is optimised for structured documents and

supports both printed and handwritten text in Bengali and English. The raw text returned by the API is then passed to a custom parser module that uses regular expressions and keyword matching to extract the transaction amount, date, and merchant name. The detected spending category is determined by matching merchant and item keywords against a predefined category taxonomy containing Bengali and English terms.

1.3 Scope and Objectives

The scope of the Taka Gelo Koi project encompasses the following functional areas:

- User authentication via Google OAuth 2.0.
- Manual and OCR-assisted transaction (expense) logging.
- Income recording from multiple sources (salary, gifts, donations, and custom).
- Monthly budget creation and threshold-based alert management.
- Savings goal tracking with allocation from income or pooled savings.
- Month-to-month savings carry-forward and savings pool management.
- Loan and debt management including repayment tracking.
- Visual spending analytics through charts and category breakdowns.
- Monthly expense reporting with category-level comparisons.
- Support for Bengali and English text in OCR processing.

The following objectives guided the development process:

- To reduce the manual effort required for expense logging by automating data extraction from receipt photographs.
- To support bilingual OCR processing for Bengali and English receipts, addressing the local context of the intended user base.
- To provide a budget management mechanism that notifies users when expenditure approaches or exceeds defined category limits.
- To present spending data through a visual dashboard that allows users to identify patterns in their financial behaviour.
- To enable users to track financial goals and manage the allocation of savings towards those goals.
- To provide a loan tracking module so users can monitor their outstanding debts and repayment history.
- To implement a secure and stateless authentication mechanism using Google OAuth 2.0.

1.4 Stakeholders

Stakeholder	Description
Primary Users	Students, salaried professionals, and small business owners who wish to monitor and manage their personal expenditures and savings goals.
Client	Rafat Bin Islam, Staff Engineer, Agoda — the project sponsor who defined the initial requirements and reviewed deliverables.
Development Team	Team NewToDev (Team 17) — a four-member team responsible for the design, implementation, and testing of the system.
Academic Supervisor	Dr. Mohammad Mahdi Hassan, Department of Computer Science and Engineering, East West University.

Chapter 2: Requirements

2.1 Requirements Collection Process

Requirements for Taka Gelo Koi were gathered through a combination of client interviews, direct user observation, and analysis of comparable personal finance tools in the Bangladeshi market. The client defined the core problem domain: users who make cash and digital payments lack a convenient way to track and categorise their expenditure, particularly when receipts are handwritten or in Bengali. The development team conducted structured interviews with representatives from the intended user groups — university students, salaried employees, and small business owners.

The team reviewed existing applications such as Wallet, Money Manager, and Monefy to identify functional gaps relevant to the local context. Requirements were prioritised using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have). High-priority requirements were those directly supporting the core expense logging, budget management, and income tracking workflows. Medium-priority requirements addressed savings goals, loan management, and analytics.

2.2 User Stories

ID	Priority	Title	Description
US-01	High	Google Sign-In	As a new user, I want to sign in using my Google account so that I can access the application without creating a separate account.
US-02	High	Log a Transaction	As a registered user, I want to manually enter an expense with amount, category, description, and date.
US-03	High	Upload Receipt for OCR	As a user, I want to upload a receipt photograph so the system extracts amount, date, and merchant automatically.
US-04	High	Auto-Categorisation	As a user, I want transactions to be automatically assigned a spending category based on the merchant or description.
US-05	High	Edit Transaction	As a user, I want to edit any existing transaction to correct inaccuracies.
US-06	High	Delete Transaction	As a user, I want to delete incorrectly entered or duplicate transaction records.
US-07	High	Set Monthly Budget	As a budget-conscious user, I want to assign a monthly spending limit to a specific category.
US-08	High	Budget Alert Threshold	As a user, I want a visual alert when my spending in a category approaches or exceeds the defined monthly limit.
US-09	High	Record Income	As a user, I want to log my income from salary, gifts, donations, or custom sources for a given month.
US-10	High	Income vs Expense Summary	As a user, I want to see total income, total spent, and net savings for any given month.
US-11	Medium	View Spending Dashboard	As a user, I want to view charts and statistics summarising my current month expenditure by category.
US-12	Medium	Monthly Analytics Report	As a user, I want a month-by-month comparison of spending to identify trends over time.
US-13	Medium	Savings Goal Tracking	As a user, I want to define savings goals with a name, target amount, icon, and optional deadline.

ID	Priority	Title	Description
US-14	Medium	Allocate to Goal	As a user, I want to allocate money from my income or savings pool towards a specific goal.
US-15	Medium	Savings Pool Management	As a user, I want unspent monthly savings to be carried forward into a total savings pool.
US-16	Medium	Loan Tracking	As a user, I want to record borrowed money with details of the lender and loan date.
US-17	Medium	Loan Repayment	As a user, I want to record repayments against a loan, deducted from my income or savings.
US-18	Low	Custom Expense Categories	As a user, I want to create custom spending categories beyond the default set.

2.3 Acceptance Criteria

US-01: Google Sign-In

- The login page presents a Google Sign-In button rendered by the @react-oauth/google library.
- Upon successful Google authentication, the backend verifies the received ID token and either creates a new user record or updates the existing one.
- The server returns the user's identifier, name, email address, and profile picture URL.
- The user's session is stored in sessionStorage and cleared automatically when the browser tab is closed.
- An appropriate error message is displayed if authentication fails.

US-02: Log a Transaction

- The transaction form accepts a numeric amount, a category from a dropdown, a text description, and a date.
- The system rejects submissions where the amount is zero or negative.
- On submission, a POST request is sent to the /transactions/ endpoint and the record is persisted.
- The transaction list and dashboard statistics update immediately upon successful creation.

US-03: Upload Receipt for OCR

- The system accepts JPEG, PNG, and WebP image files up to 10 MB.
- The uploaded image is sent to the Google Cloud Vision API using DOCUMENT_TEXT_DETECTION with Bengali and English language hints.
- Extracted text is stored against the transaction record (raw_text column).
- If the OCR service is unavailable, a descriptive error message is shown.

US-04: Auto-Categorisation

- The parser analyses extracted text and matches against a keyword taxonomy for six categories: Food, Transport, Shopping, Utilities, Entertainment, and Rent.
- If no keyword match is found, the category defaults to Other.
- A confidence score between 0 and 1 is computed for the assigned category.

US-07 and US-08: Monthly Budget and Alerts

- The budget creation form requires a category, monthly limit, and month in YYYY-MM format.

- The system prevents duplicate budgets for the same user, category, and month combination.
- A visual warning indicator appears when spending reaches the user-defined alert threshold (default: 80%).
- A distinct over-budget indicator appears when spending exceeds 100% of the limit.

US-09 and US-10: Income Tracking

- The income form accepts amount, source (Salary, Gift, Donation, or custom), optional description, and date.
- The income summary endpoint computes total income, total spent, net savings, and source breakdown for any given month.
- Carried-forward savings from previous months are included in the monthly summary.

US-13 to US-15: Savings and Goals

- A goal is created with a name, target amount, icon, colour, optional deadline, and a created month.
- Money is allocated to a goal from income (creates an associated transaction) or from the savings pool.
- A goal is automatically marked as achieved when its current_amount reaches target_amount.
- Month-end surplus income is eligible to be carried forward or pooled into total savings.

US-16 and US-17: Loan Management

- A loan record stores the original amount, remaining amount, lender name, and loan date.
- A repayment record decreases the remaining_amount on the corresponding loan.
- If the remaining amount reaches zero, the loan is marked as is_paid = True.
- Repayments can be sourced from income or from the savings pool.

2.4 Use Case Diagram

The use case diagram for Taka Gelo Koi identifies the interactions between the two primary actors — the Registered User and the Google OAuth Service — and the functional use cases of the system. Core use cases include: Sign In via Google, Log Expense, Upload Receipt (extends Log Expense), Log Income, Set Budget, View Dashboard, Track Savings Goal, Allocate to Goal, Record Loan, and Make Repayment.

Chapter 3: Architecture and Detailed Design

3.1 System Architecture

Taka Gelo Koi is structured according to a three-tier client-server architecture. This model separates the system into a presentation layer, an application layer, and a data layer, each with clearly defined responsibilities. This separation promotes maintainability, supports independent scaling of individual tiers, and reduces coupling between system components.

3.1.1 Presentation Layer

The presentation layer is a React single-page application built with the Vite toolchain. The application comprises three primary views: the Landing Page, the Login Page, and the Dashboard. Navigation between views is managed through component-level state within App.jsx using a page state variable rather than a client-side router, which simplifies the routing model for this application scope.

The Dashboard is divided into functional panels rendered conditionally based on the active tab: Overview (summary statistics and charts), Transactions (full CRUD for expenses), Budgets (monthly budget management), Income (recording and summary), Savings and Goals (goal creation, allocation, and pool management), Analytics (trend charts), Report (detailed monthly reports), and Loans (debt tracking and repayment).

A background particle animation canvas (ParticleCanvas.jsx) is rendered persistently behind all views. It creates 120 drifting particles with connecting lines, providing visual depth across all pages without obstructing the primary UI.

3.1.2 Application Layer

The application layer is implemented using the Python FastAPI framework. The server exposes a RESTful API organised into the following endpoint groups:

- Authentication: /auth/google (POST), /auth/verify/{user_id} (GET).
- Transactions: /transactions/ (POST), /transactions/{user_id} (GET), /transactions/{id} (PUT, DELETE), /summary/{user_id} (GET).
- Budgets: /budgets/ (POST), /budgets/{user_id}/{month} (GET), /budgets/{id} (PUT, DELETE), /budget-status/{user_id}/{month} (GET).
- Incomes: /incomes/ (POST), /incomes/{user_id} (GET), /incomes/{id} (PUT, DELETE), /income-summary/{user_id} (GET).
- Goals and Allocations: /goals/ (POST), /goals/{user_id} (GET), /goals/{id} (PUT, DELETE), /goal-allocations/ (POST), /goal-allocations/{user_id} (GET).
- Savings: /savings-transfers/ (POST), /savings-transfers/{user_id} (GET), /total-savings/{user_id} (GET), /check-unallocated-savings/{user_id} (GET), /confirm-auto-allocate/{user_id} (POST).
- Loans and Payments: /loans/ (POST), /loans/{user_id} (GET), /loans/{id} (PUT, DELETE), /loan-payments/ (POST), /loan-payments/{user_id} (GET).
- OCR and Parsing: /parse-image (POST), /upload-receipt/{transaction_id} (POST).
- Reporting: /monthly-report/{user_id} (GET).

3.1.3 Data Layer

The data layer consists of a PostgreSQL relational database accessed through the SQLAlchemy ORM. The database engine and session factory are configured in database.py. Database sessions are injected into FastAPI endpoints via the get_db() dependency provider function, which

ensures that sessions are created and closed cleanly for each request. The database schema spans nine tables covering all functional domains of the application.

3.2 Technology Stack

Layer	Technology	Justification
Frontend	React 18 + Vite	Component-based UI with fast HMR development server.
Frontend State	React useState / useCallback / useEffect	Lightweight state management sufficient for single-user session scope.
HTTP Client	Axios	Promise-based HTTP library with clean error handling and interceptor support.
Backend	Python FastAPI	Asynchronous, type-annotated API framework with automatic OpenAPI documentation.
ORM	SQLAlchemy	Mature Python ORM with full PostgreSQL support and SQLite for testing.
Database	PostgreSQL	Robust relational database with ACID compliance, indexing, and foreign key enforcement.
Authentication	Google OAuth 2.0	Eliminates password management; verifies identity using Google's token infrastructure.
OCR	Google Cloud Vision API	DOCUMENT_TEXT_DETECTION supports handwritten Bengali and English (1000 free images/month).
Validation	Pydantic v2	Declarative data validation tightly integrated with FastAPI for request/response schemas.
Testing Backend	pytest + SQLite	pytest for unit and integration tests; SQLite in-memory database avoids Postgres dependency.
Testing Frontend	Vitest + React Testing Library	Fast, Vite-native test runner with DOM simulation.
Styling	Custom CSS + CSS Variables	Design-token-based theming via global.css CSS custom properties.

Chapter 4: Database Design

4.1 Overview

The Taka Gelo Koi database is implemented in PostgreSQL and managed through the SQLAlchemy ORM. The schema consists of nine tables: users, transactions, budgets, incomes, goals, goal_allocations, savings_transfers, loans, and loan_payments. These tables are defined as Python model classes in models.py. A series of targeted migration scripts (migrate_google_oauth.py, migrate_budget_system.py, migrate_add_image_path.py, migrate_add_raw_text.py, migrate_income_goals.py) were applied incrementally as features were added during development.

4.2 Entity Relationship Description

The data model reflects a one-to-many relationship between the users table and all other domain tables. A single user may have zero or more records in each of the following tables: transactions, budgets, incomes, goals, goal_allocations, savings_transfers, loans, and loan_payments. Foreign key constraints (user_id referencing users.id) enforce referential integrity across all tables.

Additional inter-table relationships exist as follows: goal_allocations references goals via goal_id; loan_payments references loans via loan_id; and transactions optionally references goals via goal_id when an expense is associated with a specific savings goal. Cascade deletion is configured at the ORM level for goal_allocations (cascade on Goal) and loan_payments (cascade on Loan).

4.3 Table Definitions

Table 4.1: users

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented, indexed.
name	String	Full display name from Google OAuth token. Not null.
email	String	Unique, indexed. Used as secondary identifier.
google_id	String	Unique, indexed. Subject identifier (sub) from Google ID token. Not null.
picture	String	URL of Google profile photograph. Nullable.

Table 4.2: transactions

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented.
amount	Float	Transaction value in Taka. Enforced > 0 at the API layer.
category	String	Spending category (e.g., Food, Transport). Not null.
description	String	Free-text description or merchant name. Nullable.
date	String	YYYY-MM-DD format. Not null.
image_path	String	Relative path to uploaded receipt image. Nullable.
raw_text	String	Full OCR-extracted text from receipt. Nullable.
goal_id	Integer	FK to goals.id. Nullable — set when expense funds a goal.

Column	Data Type	Constraints / Notes
from_savings	Boolean	True if this expense was paid from the savings pool. Default: False.
user_id	Integer	FK to users.id. Associates record with its owner.

Table 4.3: budgets

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented.
user_id	Integer	FK to users.id. Not null.
category	String	Expenditure category. Not null.
monthly_limit	Float	Maximum permitted spending for category in given month.
month	String	Budget period as YYYY-MM (e.g., 2026-03).
alert_threshold	Float	% of limit at which warning triggers. Default: 80.0.
notifications_enabled	Boolean	Controls whether threshold alert is active. Default: True.

Table 4.4: incomes

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented.
user_id	Integer	FK to users.id. Not null.
amount	Float	Income value in Taka. Not null.
source	String	Source label: Salary, Gift, Donation, or custom. Not null.
description	String	Optional free-text description. Nullable.
date	String	YYYY-MM-DD format. Not null.

Table 4.5: goals

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented.
user_id	Integer	FK to users.id. Not null.
name	String	Goal name. Not null.
target_amount	Float	Target savings amount. Not null.
current_amount	Float	Amount saved so far. Default: 0.0.
icon	String	Emoji icon for display. Default: Emoji target symbol.
color	String	Hex colour for goal card. Default: #e8b84b.
deadline	String	Optional target date in YYYY-MM-DD. Nullable.
status	String	active or achieved. Default: active.
created_month	String	YYYY-MM month when goal was created. Not null.

Table 4.6: goal_allocations

Column	Data Type	Constraints / Notes
id	Integer	Primary key.
goal_id	Integer	FK to goals.id. Not null.
user_id	Integer	FK to users.id. Not null.
amount	Float	Amount allocated to the goal. Not null.
month	String	YYYY-MM month when allocation occurred.
source	String	income or savings. Default: income.
notes	String	Optional notes. Nullable.

Table 4.7: savings_transfers

Column	Data Type	Constraints / Notes
id	Integer	Primary key.
user_id	Integer	FK to users.id. Not null.
amount	Float	Amount transferred. Not null.
from_month	String	Source month YYYY-MM. Not null.
to_month	String	Destination month YYYY-MM. Null if added to pool.
transfer_type	String	carry_forward, pool, or loan_repayment.
notes	String	Optional notes. Nullable.

Table 4.8: loans

Column	Data Type	Constraints / Notes
id	Integer	Primary key.
user_id	Integer	FK to users.id. Not null.
amount	Float	Original loan amount. Not null.
remaining_amount	Float	Outstanding balance; decreases with repayments. Not null.
description	String	Optional loan description. Nullable.
lender	String	Person or institution name. Nullable.
date	String	YYYY-MM-DD when loan was taken. Not null.
is_paid	Boolean	True when fully repaid. Default: False.

Table 4.9: loan_payments

Column	Data Type	Constraints / Notes
id	Integer	Primary key.
loan_id	Integer	FK to loans.id. Not null.
user_id	Integer	FK to users.id. Not null.
amount	Float	Repayment amount. Not null.
source	String	income or savings. Not null.

Column	Data Type	Constraints / Notes
month	String	YYYY-MM when repayment was made.
date	String	YYYY-MM-DD repayment date. Not null.
notes	String	Optional notes. Nullable.

4.4 Indexing Strategy

The users table maintains indexes on email and google_id, which are the primary lookup keys during authentication. The id column on every table is indexed as primary key. The budgets table carries a unique composite index on (user_id, category, month) to enforce the constraint that a single budget entry exists per user per category per month. This index is created in migrate_budget_system.py as idx_budgets_user_category_month.

4.5 Entity Relationship Diagram

The ER diagram shows nine entities connected through a central users entity. users has one-to-many relationships with transactions, budgets, incomes, goals, goal_allocations, savings_transfers, loans, and loan_payments. goals is additionally related to goal_allocations (one-to-many) and transactions (optional FK). loans is related to loan_payments (one-to-many with cascade delete).

Chapter 5: Class Design

5.1 Overview

The class design of Taka Gelo Koi is described across four functional layers: database model classes, Pydantic schema classes, service modules, and React component/hook units. The following sections describe the responsibilities and key members of each primary class or module.

5.2 Backend Classes and Modules

5.2.1 Database Model Classes (models.py)

The models.py module defines nine SQLAlchemy declarative model classes. Each class maps directly to a database table and defines its columns using SQLAlchemy's Column construct.

- User: Represents an authenticated user. Attributes: id, name, email, google_id, picture. Has one-to-many bidirectional relationships with Transaction, Budget, Income, Goal, GoalAllocation, SavingsTransfer, Loan, and LoanPayment.
- Budget: Monthly spending limit for a category. Attributes: user_id (FK), category, monthly_limit, month, alert_threshold, notifications_enabled.
- Transaction: A single expense record. Attributes: user_id (FK), amount, category, description, date, image_path, raw_text, goal_id (nullable FK), from_savings.
- Income: An income record. Attributes: user_id (FK), amount, source, description, date.
- Goal: A savings or financial goal. Attributes: user_id (FK), name, target_amount, current_amount, icon, color, deadline, status, created_month. Has one-to-many with GoalAllocation (cascade delete).
- GoalAllocation: Money allocated to a goal. Attributes: goal_id (FK), user_id (FK), amount, month, source, notes.
- SavingsTransfer: Record of savings carried forward or pooled. Attributes: user_id (FK), amount, from_month, to_month, transfer_type, notes.
- Loan: A borrowed-money record. Attributes: user_id (FK), amount, remaining_amount, description, lender, date, is_paid. Has one-to-many with LoanPayment (cascade delete).
- LoanPayment: A repayment against a loan. Attributes: loan_id (FK), user_id (FK), amount, source, month, date, notes.

5.2.2 Pydantic Schema Classes (schemas.py)

Pydantic schemas enforce input validation and define API request/response shapes. Key classes:

- GoogleTokenRequest / GoogleLoginResponse: Authentication request and response.
- TransactionCreate / TransactionOut / TransactionUpdate: CRUD schemas for expenses.
- BudgetCreate / BudgetOut / BudgetUpdate / BudgetStatus: Budget management schemas. BudgetStatus carries computed fields: spent, remaining, percentage_used, is_over_budget, is_near_limit.
- IncomeCreate / IncomeOut / IncomeUpdate: Income record schemas.
- GoalCreate / GoalOut / GoalUpdate: Savings goal schemas.
- GoalAllocationCreate / GoalAllocationOut: Goal allocation schemas.
- SavingsTransferCreate / SavingsTransferOut: Savings transfer schemas.
- LoanCreate / LoanOut / LoanUpdate: Loan record schemas.
- LoanPaymentCreate / LoanPaymentOut: Loan repayment schemas.
- AutoAllocateConfirm: Accepts a list of month strings for auto-allocation confirmation.

5.2.3 OCR Service Module (ocr_service.py)

- GoogleVisionConfig (class): Holds API key (from environment variable GOOGLE_VISION_API_KEY), language hints (["bn", "en"]), and REST endpoint URL.
- is_paddleocr_available(): Returns boolean indicating whether API key is configured.
- extract_text_from_image(image_path): Encodes image as Base64, POSTs to Vision API, returns extracted text string. Raises specific exceptions for file-not-found, missing key, timeout, and API errors.
- extract_text_from_image_bytes(image_buffer): Memory-only variant accepting a BytesIO buffer.
- extract_text_with_details(image_path, min_confidence): Returns full text plus per-word confidence scores and aggregate statistics.
- get_ocr_info(): Returns engine metadata, language support, availability, and pricing information.

5.2.4 Parser Service Module (parser_service.py)

- normalize_digits(text): Converts Bengali numerals (০১২৩৪৫৬৭৮৯) to ASCII equivalents.
- extract_amount(text): Applies prioritised regex patterns for Bengali total keywords, Taka symbols, and English total labels. Returns (amount, confidence).
- extract_date(text): Applies DD/MM/YYYY, YYYY-MM-DD, and English month-name patterns. Returns (ISO date string, confidence).
- extract_merchant(text): Scans first four receipt lines, filtering phone numbers and address patterns. Returns (merchant name, confidence).
- detect_category(text): Keyword matching against Bengali + English taxonomy for Food, Transport, Shopping, Utilities, Entertainment, and Rent. Returns (category, confidence).
- parse_receipt(text): Orchestrates the four extraction functions and returns a unified dictionary with amount, date, merchant, category, and confidence sub-keys.

5.3 Frontend Components and Hooks

5.3.1 App.jsx

App.jsx is the root component. It maintains page state (landing, login, dashboard) and user state. States are initialised from sessionStorage on mount. The component renders ParticleCanvas persistently, then switches between LandingPage, LoginPage, and Dashboard based on page state.

5.3.2 useApi.js (Custom Hook)

The useApi hook is the complete data access layer for the dashboard. It accepts the authenticated user ID and exposes a comprehensive API surface:

- State variables: transactions, budgets, summary, incomes, incomeSummary, goals, achievedGoals, goalAllocations, savingsTransfers, totalSavingsPool, loans, loanPayments, loading, cats.
- Fetchers: fetchTransactions, fetchSummary, fetchBudgets, fetchIncomes, fetchIncomeSummary, fetchGoals, fetchGoalAllocations, fetchSavingsTransfers, fetchTotalSavings, fetchLoans, fetchLoanPayments, fetchMonthlyReport.
- Mutations: addTransaction, deleteTransaction, updateTransaction, createBudget, deleteBudget, updateBudget, addIncome, deleteIncome, updateIncome, addGoal, updateGoal, deleteGoal, allocateToGoal, createSavingsTransfer, addLoan, deleteLoan, addLoanPayment.
- Auto-allocate: checkUnallocatedSavings (checks if prior months have unhandled surplus), confirmAutoAllocate (sends selected months to backend for processing).

- The cats (categories) state is persisted to sessionStorage per user ID, allowing custom categories without a database endpoint.

5.3.3 Page and Layout Components

- LandingPage.jsx: Marketing page with navbar, hero, statistics band, feature grid, how-it-works walkthrough, testimonials, and CTA. Uses IntersectionObserver for scroll animations.
- LoginPage.jsx: Two-panel authentication page. Left panel has animated floating data cards. Right panel renders the Google Sign-In button via @react-oauth/google.
- Dashboard.jsx: Primary application view. Renders sidebar navigation, topbar, and eight tab panels. Consumes the useApi hook entirely.
- ParticleCanvas.jsx: HTML5 canvas-based particle animation. Creates 120 particles with connecting lines. Uses requestAnimationFrame and handles window resize.
- GoogleAuth.jsx: Standalone Google OAuth component supporting One-Tap login. Stores user data in localStorage. Usable independently of the main application flow.

5.4 Class Diagram

The class diagram depicts the nine SQLAlchemy model classes with their attributes and relationships, alongside the corresponding Pydantic schema classes. Backend service modules (ocr_service, parser_service) are shown as utility classes with their public function signatures. Frontend components are represented with their props and state variables, and useApi.js is shown as a custom hook class exposing all state and function members.

Chapter 6: Design Patterns

6.1 Overview

The Taka Gelo Koi system applies two established software design patterns from the Gang of Four catalogue: the Factory Method pattern and the Adapter pattern. These patterns were selected to address specific structural challenges in the codebase — namely, database session lifecycle management and the integration of an external third-party API. This chapter describes each pattern, its general purpose, and provides the core implementation code from the project.

6.2 Factory Method Pattern (Creational)

Purpose

The Factory Method pattern provides an interface for creating objects in a superclass but allows subclasses or frameworks to alter the type of objects that will be created. Its primary objective is to encapsulate instantiation logic, ensuring that objects are created with a consistent configuration without exposing complex initialisation code to client components.

Application in Taka Gelo Koi

The Factory Method pattern is applied in database.py for database session lifecycle management. Rather than having each FastAPI endpoint independently instantiate a database connection with hardcoded configuration, the system defines a SessionLocal factory created by SQLAlchemy's sessionmaker. The get_db() dependency generator further abstracts this by acting as the factory method that creates, yields, and closes database sessions cleanly for each HTTP request.

Core Implementation — database.py (Factory Method Pattern)

```
# Factory: sessionmaker creates a configured session factory
engine = create_engine(SQLALCHEMY_DATABASE_URL)

# SessionLocal is the factory — not an instance
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Factory Method: get_db() produces a new session for each request
def get_db():
    db = SessionLocal()    # Factory creates the object
    try:
        yield db           # Client component receives the product
    finally:
        db.close()         # Factory cleans up the product
```

When a FastAPI endpoint declares a dependency on get_db(), FastAPI's dependency injection system calls this factory method and injects the produced Session object into the endpoint function. The endpoint never needs to know the configuration details (autocommit, autoflush, engine binding). This decouples the application layer from the data persistence configuration and supports seamless substitution during testing — the test suite replaces SessionLocal with a SQLite-backed factory without changing any endpoint code.

6.3 Adapter Pattern (Structural)

Purpose

The Adapter pattern acts as a structural bridge between two incompatible interfaces. An intermediary component translates requests from a client into a format that a distinct, often third-

party, service can process. It allows disparate classes or external systems to work together without modifying their existing source code.

Application in Taka Gelo Koi

The Adapter pattern is implemented in ocr_service.py to integrate the Google Cloud Vision API. The external API requires Base64-encoded image content, a deeply nested JSON payload specifying detection features and language hints, and returns a complex nested response. The internal application only needs a plain text string. ocr_service.py acts as the Adapter: it exposes a simple function interface while internally handling all protocol translation.

Core Implementation — ocr_service.py (Adapter Pattern)

```
class GoogleVisionConfig:
    API_KEY: str = os.environ.get("GOOGLE_VISION_API_KEY", "")
    LANGUAGE_HINTS = ["bn", "en"]
    ENDPOINT = (
        "https://vision.googleapis.com/v1/images:annotate"
        "?key={api_key}"
    )

def extract_text_from_image(image_path: str) -> str:
    # Step 1: Encode image to Base64 (API requirement)
    with open(image_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    # Step 2: Build the nested payload the API expects
    payload = {
        "requests": [{
            "image": {"content": image_data},
            "features": [{"type": "DOCUMENT_TEXT_DETECTION"}],
            "imageContext": {
                "languageHints": GoogleVisionConfig.LANGUAGE_HINTS
            }
        }]
    }

    # Step 3: Execute HTTP request
    url = GoogleVisionConfig.ENDPOINT.format(
        api_key=GoogleVisionConfig.API_KEY)
    response = requests.post(url, json=payload, timeout=30)
    response.raise_for_status()
    data = response.json()

    # Step 4: Extract plain text from the nested response
    first = data.get("responses", [{}])[0]
    full_annotation = first.get("fullTextAnnotation", {})
    if full_annotation.get("text"):
        return full_annotation["text"].strip()
    text_annotations = first.get("textAnnotations", [])
    if text_annotations:
        return text_annotations[0].get("description", "").strip()
    return ""
```

The core application in main.py only ever calls extract_text_from_image(path), receiving a plain string in return. It is entirely isolated from the Base64 encoding, JSON payload structure, HTTP layer, and nested response parsing. This allows the underlying OCR provider to be changed (for example, to a local model) simply by replacing the ocr_service.py implementation without touching any other module.

Chapter 7: UI/GUI Design

7.1 Design Philosophy

The Taka Gelo Koi user interface is designed around clarity and minimal cognitive load. A dark colour theme uses a deep navy background (#030c23) with a gold accent colour (#e8b84b) for emphasis and interactive elements. This palette maintains sufficient contrast for readability while establishing a professional visual identity. A global CSS variable system defined in global.css serves as the single source of truth for the colour palette, typography, and spacing constants across all components.

7.2 Typography

Three typefaces are used across the interface: Playfair Display (serif) for headings and display text; Inter (sans-serif) as the primary body typeface for all instructional and data text; and Space Mono (monospace) for numeric values, dates, category labels, and other data-dense fields.

7.3 Landing Page

The landing page is the first point of contact for prospective users. Its sections include:

- Navigation Bar: Fixed at the top, containing the Taka Gelo Koi logo, section links (Features, How It Works, Reviews), and Login and Get Started buttons. The bar becomes opaque with a backdrop blur when scrolled.
- Hero Section: Primary value proposition headline, supporting description, two CTA buttons, and a three-dimensional CSS-transformed mock dashboard card showing sample financial data. Three floating badge elements animate continuously.
- Statistics Band: Four headline metrics — 1k+ active users, 2M+ BDT tracked monthly, 98% satisfaction, Free forever.
- Features Section: Six feature cards in a responsive grid covering expense logging, income tracking, visual analytics, smart budget alerts, savings goals, and Google Sign-In.
- How It Works: Four-step walkthrough with a mock transaction list screen.
- Reviews Section: Three user testimonials.
- Call-to-Action: Final conversion section linking to the login page.

7.4 Login Page

A two-panel layout. The left panel presents animated floating information cards showing sample financial statistics (Monthly Report, Budget Status, Savings Goal progress) plus a user testimonial quote, grid overlay, and three radial gradient orb effects. The right panel contains the Google Sign-In button rendered by @react-oauth/google, three trust badges (Encrypted, OAuth 2.0, GDPR Compliant), and an inline error display for failed authentication attempts.

7.5 Dashboard

The dashboard adopts a two-column layout consisting of a persistent left sidebar and a main content area.

Sidebar

Displays the Taka Gelo Koi logo, a grouped navigation menu (Main: Overview, Transactions, Budgets, Income; Insights: Savings and Goals, Analytics, Report, Loans), and a footer showing the user's profile photograph, name, email, and logout button.

Overview Panel

Four statistical cards: total spending for the current month, total transaction count, average transaction value, and active budget count. Below: a pie chart of spending by category, a bar chart of the last seven days' transactions, and the five most recent transactions.

Transactions Panel

Full transaction history in a scrollable list. Each item shows a category icon, description, category tag, date, and amount with inline edit and delete controls. An Add Transaction form includes amount, category, description, date, and optional receipt image upload.

Budgets Panel

All budgets for the current month displayed as progress bars coloured green (normal), orange (alert threshold reached), or red (over budget). A budget creation form requires category, monthly limit, and optional alert threshold percentage.

Income Panel

Income log with source, amount, and date. An income summary card shows total income, total spent, net savings, and source breakdown for the selected month.

Savings and Goals Panel

Active goals displayed as cards with icon, name, progress bar (current / target), deadline, and allocate button. A savings pool summary shows total pooled savings, amounts spent on goals and loans, and net remaining pool. A create-goal form and an allocate-to-goal form are embedded in the panel.

Analytics Panel

Grid of charts covering monthly spending trends, category distribution, income versus expense comparison, and savings growth over time.

Report Panel

Month selector control with a detailed category breakdown showing budget versus actual spending for the selected month.

Loans Panel

Active and paid loans listed with lender name, original amount, remaining balance, and a repayment form. Repayments are sourced from income or savings pool.

7.6 Responsive Design

Media queries in Dashboard.css and LandingPage.css apply layout adjustments for screens narrower than 900 pixels. At this breakpoint, the sidebar collapses on the dashboard, multi-column grids become single-column, the hero section stacks vertically, and the mock dashboard card is hidden. The login page restructures from horizontal two-panel to vertically stacked. This ensures usability across mobile and tablet devices.

7.7 UI Screenshots

Screenshots of the landing page, login page, dashboard overview, transactions panel, budgets panel, income panel, savings and goals panel, and loans panel are included as Figures 7.1 through 7.8 in the accompanying appendix materials.

Chapter 8: Scheduling and Estimation

8.1 Project Timeline

The Taka Gelo Koi project was executed over a 14-week period from January 2026 to April 2026. The schedule was divided into four phases:

Phase	Duration	Key Activities
Phase 1: Requirements and Design	Weeks 1–3 (Jan 2026)	Requirements gathering, user story definition, architecture design, database schema design, technology selection.
Phase 2: Core Development	Weeks 4–8 (Feb 2026)	Backend setup (FastAPI, PostgreSQL, Google OAuth), transaction and budget CRUD, OCR integration, landing page and login UI.
Phase 3: Feature Expansion	Weeks 9–12 (Mar 2026)	Income tracking, savings goals, loan management, savings pool, auto-allocation, enhanced dashboard panels, analytics.
Phase 4: Testing and Finalisation	Weeks 13–14 (Apr 2026)	Unit and integration testing, bug fixes, final UI polish, documentation, report writing.

8.2 KLOC Estimation

Lines of code were counted across all backend and frontend source files (excluding blank lines and pure comment lines):

Module / File	Category	SLOC (approx.)
main.py	Backend — FastAPI routes	680
models.py	Backend — ORM models	105
database.py	Backend — DB configuration	35
schemas.py	Backend — Pydantic schemas	210
ocr_service.py	Backend — OCR service	245
parser_service.py	Backend — Parser service	205
Migration scripts (5 files)	Backend — Database migrations	290
conftest.py	Test — Fixtures	70
test_api_routes.py	Test — API integration	225
test_schemas.py	Test — Schema validation	200
App.jsx + main.jsx	Frontend — Root	65
Dashboard.jsx	Frontend — Dashboard	950
useApi.js	Frontend — Data hook	325
LandingPage.jsx	Frontend — Landing page	165
LoginPage.jsx	Frontend — Login page	125
ParticleCanvas.jsx	Frontend — Animation	55
GoogleAuth.jsx	Frontend — Auth component	110

Module / File	Category	SLOC (approx.)
global.css + App.css	Frontend — Global styles	85
Dashboard.css	Frontend — Dashboard styles	380
LandingPage.css	Frontend — Landing styles	280
LoginPage.css	Frontend — Login styles	175
TOTAL		~4,980

Total project size: approximately 4,980 SLOC, equivalent to 4.98 KLOC (~5 KLOC). This figure encompasses all production source code and test files but excludes configuration files (package.json, vite.config.js, .env templates).

8.3 Function Point Estimation

Function Point Analysis (FPA) was applied using the IFPUG method. Each functional component was classified and assigned an unadjusted complexity weight.

FP Component	Count	Avg. Weight	Unadjusted FP
External Inputs (EI)	14	4	56
External Outputs (EO)	8	5	40
External Inquiries (EQ)	6	4	24
Internal Logical Files (ILF)	9	10	90
External Interface Files (EIF)	2	7	14
TOTAL (UAF)			224

External Inputs (14): Login via Google, Add/Edit/Delete Transaction, Add/Edit/Delete Budget, Add/Edit/Delete Income, Add Goal, Allocate to Goal, Create Savings Transfer, Add Loan, Add Loan Payment, Confirm Auto-Allocate, Upload Receipt.

External Outputs (8): Dashboard overview, Budget status cards, Income summary, Goal progress cards, Savings pool summary, Monthly report, Analytics charts, OCR text result.

External Inquiries (6): Transaction list, Budget list, Income list, Goal list, Loan list, Monthly summary.

Internal Logical Files (9): users, transactions, budgets, incomes, goals, goal_allocations, savings_transfers, loans, loan_payments.

External Interface Files (2): Google OAuth API, Google Cloud Vision API.

Technical Complexity Factor (TCF): Fourteen value adjustment factors were assessed with scores from 0 to 5. Relevant high-weight factors include: distributed data processing (4), performance requirements (3), online data entry (5), end-user efficiency (4), online update (5), reusability (3). Total DI = 46. VAF = 0.65 + (0.01 × 46) = 1.11.

Adjusted Function Points (AFP) = UAF × VAF = 224 × 1.11 = 248.64 ≈ 249 FP.

8.4 Effort and Duration Estimation (COCOMO II)

The COCOMO II Post-Architecture model was applied using the project KLOC estimate of 5 KLOC. The project was classified in the semi-detached mode (a small team with mixed experience and moderate familiarity with the tools and domain).

Parameter	Value	Notes
Size (KLOC)	5.0	Total estimated SLOC / 1000
Mode	Semi-detached	Small team, moderate domain familiarity
Effort coefficient (a)	3.0	COCOMO II semi-detached constant
Effort exponent (b)	1.12	COCOMO II semi-detached constant
Duration coefficient (c)	2.5	COCOMO II constant
Duration exponent (d)	0.35	COCOMO II constant

Effort (E) = a × (KLOC)^b = 3.0 × (5.0)^1.12 = 3.0 × 6.10 = 18.3 person-months.

Development Duration (D) = c × E^d = 2.5 × (18.3)^0.35 = 2.5 × 2.97 = 7.4 months.

Average Team Size = E / D = 18.3 / 7.4 = 2.5 persons (aligns with a team of 4 where members work part-time across coursework obligations).

Actual project execution used a four-person team over approximately 14 weeks (3.5 months). With full-time equivalent contribution estimated at roughly 40% of working hours per member (due to concurrent coursework), the effective effort aligns closely with the COCOMO II estimate.

8.5 Productivity Metrics

Metric	Value
Total SLOC	~4,980
Adjusted Function Points (AFP)	249
SLOC per FP	~20
Estimated Effort (COCOMO II)	18.3 person-months
Actual Duration	3.5 months (14 weeks)
Team Size	4 members
Productivity	~50 FP / person-month
Defect Density (from testing)	< 1 defect per KLOC (post-test)

Chapter 9: Testing

9.1 Testing Strategy

The testing strategy for Taka Gelo Koi covers both the Python backend and the React frontend. Backend tests are implemented using the pytest framework with an in-memory SQLite database to avoid reliance on a production PostgreSQL instance. Frontend tests use Vitest and the React Testing Library, with Google OAuth components and Axios network calls mocked to prevent real network requests. All tests are designed to execute in any environment without external service access.

Tests are categorised as follows: unit tests verify the behaviour of individual functions or modules in isolation (schema validation, parser functions), and integration tests verify the behaviour of API endpoints through the full request–validation–database–response cycle using FastAPI's TestClient.

9.2 Backend Test Coverage

Schema Validation Tests (test_schemas.py)

Contains 25 tests covering five Pydantic schema classes: GoogleTokenRequest, TransactionCreate, TransactionUpdate, BudgetCreate, and BudgetUpdate. Test categories:

- Required fields: Missing required fields raise pydantic.ValidationError.
- Type coercion: Integer amount values are coerced to float without error.
- Type rejection: Non-numeric string values for amount fields raise ValidationError.
- Optional fields: Optional fields correctly default to their configured values (e.g., alert_threshold defaults to 80.0, notifications_enabled defaults to True).
- Partial update schemas: TransactionUpdate and BudgetUpdate allow all fields to be None simultaneously.

API Route Tests (test_api_routes.py)

Contains 22 integration tests exercising FastAPI endpoints through TestClient. The test database is an in-memory SQLite file (test_tye.db) reset between test class setups. Test classes and coverage:

- TestHealthEndpoints (4 tests): Verifies that / returns HTTP 200 with {"status": "running"} and /health returns HTTP 200 with {"database": "connected"}.
- TestTransactionEndpoints (10 tests): Full CRUD lifecycle — successful creation, rejection of negative and zero amounts, 422 for missing required fields, empty list for new user, filtering by user ID, update by ID, 404 for non-existent update/delete, and rejection of negative update amounts.
- TestSummaryEndpoint (5 tests): Empty total for new user, correct total aggregation, month filtering (February transactions must not appear in March summary), category breakdown dictionary, and month field in response.
- TestBudgetEndpoints (4 tests): Successful budget creation, duplicate budget rejection (same user/category/month), retrieval by month, and budget deletion.

9.3 Frontend Test Coverage

Component and Hook Tests (App.test.jsx)

Contains 30 tests covering the React application:

- App routing: Correct page renders for landing, login, and dashboard states. Session restoration from sessionStorage on mount.
- LandingPage: Hero text content, Get Started button triggers navigation, feature cards render correctly.
- LoginPage: Google Sign-In button renders, successful login calls onLogin with normalised user data, network errors display inline error message.
- useApi hook: toYM utility converts dates correctly, initial state values are empty arrays and zero totals, fetchTransactions populates the transactions state, addTransaction triggers a POST and refetches.

9.4 Test Results Summary

Test File	Test Count	Pass	Fail	Coverage Area
test_schemas.py	25	25	0	Pydantic schema validation, default values, type coercion.
test_api_routes.py	22	22	0	FastAPI endpoint integration, CRUD, validation, month filtering.
App.test.jsx	30	30	0	React routing, component rendering, useApi hook, auth flow.
TOTAL	77	77	0	All backend and frontend test cases combined.

9.5 Test Configuration

The conftest.py file defines the following session-scoped and function-scoped pytest fixtures used across all backend test files:

- test_engine: Session-scoped SQLite engine. Creates all ORM tables once at the start of the test run and drops them at the end.
- db_session: Function-scoped database session. Wraps each test in a transaction that is rolled back after the test completes, ensuring full isolation between tests without truncating tables.
- sample_user: A User object pre-committed to the test database for tests requiring a foreign key dependency.
- sample_transaction: A Transaction tied to sample_user for tests requiring an existing record.
- sample_budget: A Budget tied to sample_user for budget-specific tests.

Appendix

A. Project Resources

1. Project Information Drive: Google Drive Folder — Taka Gelo Koi
2. Source Code Repository: GitHub — Taka Gelo Koi (formerly TYE)
3. Initial Project Proposal: View Proposal Document

B. Database Migration History

Migration Script	Purpose	Order
migrate_google_oauth.py	Adds google_id and picture columns to users; removes password_hash.	1st
migrate_budget_system.py	Creates budgets table with indexes; converts transactions.amount to FLOAT.	2nd
migrate_add_image_path.py	Adds image_path column to transactions.	3rd
migrate_add_raw_text.py	Adds raw_text column to transactions for OCR extracted text.	4th
migrate_income_goals.py	Creates incomes, goals, goal_allocations, savings_transfers tables; adds goal_id and from_savings to transactions.	5th

C. API Endpoint Reference

Method	Endpoint	Description
POST	/auth/google	Verify Google ID token; create or retrieve user.
POST	/transactions/	Create a new expense transaction.
GET	/transactions/{user_id}	List all transactions for a user.
PUT	/transactions/{id}	Update a transaction by ID.
DELETE	/transactions/{id}	Delete a transaction by ID.
GET	/summary/{user_id}?month=	Monthly spending total and category breakdown.
POST	/budgets/	Create a monthly budget.
GET	/budgets/{user_id}/{month}	List budgets for a user and month.
PUT	/budgets/{id}	Update a budget.
DELETE	/budgets/{id}	Delete a budget.
POST	/incomes/	Record an income entry.
GET	/incomes/{user_id}	List all income records for a user.
GET	/income-summary/{user_id}?month=	Income vs expense summary for a month.
POST	/goals/	Create a savings goal.
GET	/goals/{user_id}?status_filter=	List goals (filter: active or achieved).
POST	/goal-allocations/	Allocate funds to a goal.
GET	/total-savings/{user_id}	Aggregated savings pool totals.

Method	Endpoint	Description
POST	/savings-transfers/	Record a savings carry-forward or pool transfer.
GET	/check-unallocated-savings/{user_id}	Check months with unhandled surplus savings.
POST	/confirm-auto-allocate/{user_id}	Auto-allocate surplus savings for specified months.
POST	/loans/	Record a new loan.
GET	/loans/{user_id}	List all loans for a user.
POST	/loan-payments/	Record a loan repayment.
GET	/monthly-report/{user_id}?month=	Detailed monthly category report.
POST	/parse-image	OCR parse an uploaded receipt image.
GET	/health	Application health check endpoint.

D. OCR Configuration

Parameter	Value
Engine	Google Cloud Vision API
Detection Type	DOCUMENT_TEXT_DETECTION
Language Hints	bn (Bengali), en (English)
Free Tier	1,000 images per month
Cost After Free Tier	USD 1.50 per 1,000 images
Supported Formats	JPEG, PNG, WebP, TIFF, GIF, BMP, PDF, WEBP
Estimated Accuracy	85–95% for handwritten Bengali + English receipts